

Национальный исследовательский университет ИТМО

Факультет безопасности информационных технологий

Алгоритмы и структуры данных

Лабораторная работа №2

«Цифровая сортировка с использованием кольцевой
очереди»

Выполнил:

Студент группы N3247

Сивоконь А.А.

(подпись)

Преподаватель:

Ерофеев С.А.

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026

Содержание

1	Постановка задачи	2
2	Описание алгоритма сортировки	3
2.1	Кольцевая очередь на односвязном списке	3
2.2	Алгоритм цифровой сортировки (Radix Sort LSD)	3
2.3	Сравнение с реализацией через массив (ЛР 1)	5
2.4	Маршрут программы	6
3	Описание используемых переменных	7
4	Тестирование программы	9
4.1	Тест 1: стандартный набор из 20 элементов	9
4.2	Тест 2: малый набор из 8 элементов	9
4.3	Тест 3: несуществующий файл	10
4.4	Тест 4: пустой файл	10
5	Блок-схемы алгоритма	11
6	Исходный код программы	12
6.1	Файл main.c	12
6.2	Файл radix_sort.h	14
6.3	Файл radix_sort.c	14
	Заключение	19
	Приложение А. Блок-схемы алгоритма	20

1. Постановка задачи

Цель работы: разработать программу цифровой сортировки (Radix Sort LSD) для последовательности целых неотрицательных чисел, считываемых из файла, используя в качестве вспомогательной структуры данных **кольцевую очередь на основе односвязного списка** вместо вспомогательного массива; определить сложность программы.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Реализовать кольцевую очередь на односвязном списке с операциями `enqueue`, `dequeue`, `queue_is_empty`, `queue_free`;
2. Реализовать чтение массива чисел из файла;
3. Реализовать алгоритм цифровой сортировки LSD через 10 кольцевых очередей-корзин;
4. Реализовать функцию вывода массива на экран;
5. Оценить теоретическую сложность алгоритма;
6. Провести тестирование программы на различных наборах данных;
7. Составить блок-схемы алгоритма и оформить отчёт.

2. Описание алгоритма сортировки

2.1. Кольцевая очередь на односвязном списке

В качестве вспомогательной структуры данных используется кольцевая очередь на основе динамически выделяемых узлов:

- Узел **Node** содержит значение **value** и указатель **next** на следующий узел.
- Очередь **Queue** хранит указатель **tail** на *хвостовой* узел; голова очереди доступна как **tail->next**. Такая организация обеспечивает $O(1)$ как для вставки (**enqueue**), так и для извлечения (**dequeue**).
- При добавлении первого элемента узел замыкается на себя: **node->next = node**.

Операции и их сложность:

Функция	Описание	Операций
queue_init	инициализировать поля	7
queue_is_empty	проверить size==0	4
enqueue	добавить в хвост	29 (не пустая); 20 (пустая)
dequeue	снять с головы	28 (много эл.); 24 (один эл.)
queue_free	освободить все узлы	$6n + 16$ ($n \geq 1$)

2.2. Алгоритм цифровой сортировки (Radix Sort LSD)

Цифровая сортировка — нерекурсивный алгоритм, обрабатывающий числа поразрядно. В варианте LSD обработка ведётся от *младшего* разряда к *старшему*.

Пошаговый алгоритм:

1. Нахождение максимального элемента.

Функция **find_max** обходит весь массив и возвращает наибольшее значение. По нему определяется число разрядов d .

2. Цикл по разрядам ($\text{exp} = 1, 10, 100, \dots$).

Пока $\text{max_val} / \text{exp} > 0$, выполняется один проход сортировки по текущему разряду.

3. Один проход (`counting_sort_by_queue`):

- (a) **Шаг 1.** Инициализировать 10 кольцевых очередей `buckets[0..9]`.
- (b) **Шаг 2.** Для каждого элемента вычислить цифру разряда: $(\text{arr}[i] / \text{exp}) \% 10$; выполнить `enqueue` в соответствующую корзину.
- (c) **Шаг 3.** Обходя корзины `buckets[0..9]` по порядку, последовательно извлекать элементы через `dequeue` и записывать обратно в `arr[]`. Порядок FIFO внутри каждой корзины гарантирует устойчивость сортировки.
- (d) **Шаг 4.** Освободить все 10 очередей (`queue_free`).

Оценка сложности алгоритма:

- Временная: $O(d \cdot n)$, где n — количество элементов, d — число разрядов в максимальном числе. При фиксированном d сложность **линейная**: $O(n)$.
- Пространственная: $O(n)$ — на одном проходе суммарно выделяется n узлов `Node`; после сбора они освобождаются.

Точный подсчёт с учётом всех 7 типов операций (вызов метода, выход из метода, переход по указателю (`->`), присваивание, сравнение, арифметика, индексация). Вызов вспомогательной функции считается как 1 операция; её внутренние операции к вызывающей не добавляются.

$$T_{find_max} = 7n - 2 \quad ()$$

Разбивка одного прохода `counting_sort_by_queue` по шагам ($k = 10$):

Шаг / действие	Операций
Шаг 1: $j=0$ (1); $j<10$ (11 сравн.); $j++$ (10×2); <code>&buckets[j]</code> (10); вызов <code>queue_init</code> (10)	52
Шаг 2: заголовок цикла i ($3n+2$); тело: <code>arr[i] + /exp + %BASE + digit= + &buckets[digit] + arr[i] +</code> вызов <code>enqueue + !=0</code> ($8 \times n$)	$11n + 2$
Шаг 3: $j=0$ ($1+1$); $j<10$ (11 сравн.); $j++$ (10×2); проверка <code>while is_empty</code> ($3 \times (n+10)$); тело <code>dequeue</code> ($3 \times n$); <code>idx++</code> ($2 \times n$)	$8n + 63$
Шаг 4: $j=0$ (1); $j<10$ (11); $j++$ (10×2); <code>&buckets[j]</code> (10); вызов <code>queue_free</code> (10)	52
Итого	$19n + 169$

$$T_1 = 19n + 169 \quad (k = 10)$$

$$T_{radix_sort} = \underbrace{(7n - 2)}_{find_max} + \underbrace{(7 + 5d)}_{find_max} + d \cdot (19n + 169) = 7n + 5 + d \cdot (19n + 174)$$

Асимптотически: $T = O(d \cdot n)$, при фиксированных d и $k = 10$ — $O(n)$.

2.3. Сравнение с реализацией через массив (ЛР 1)

Характеристика	ЛР 1 (массив)	ЛР 2 (очереди)
Один проход	$26n + 120$	$19n + 169$
Доп. память	$O(n)$ массив + $O(k)$ count	$O(n)$ узлов Node
Кол-во <code>malloc</code>	1 раз за проход	n раз за проход
Устойчивость	да (обход справа)	да (порядок FIFO)
Асимптотика	$O(d \cdot n)$	$O(d \cdot n)$

Реализация через очереди имеет меньший линейный коэффициент на один проход ($19n$ против $26n$), но большую константу (169 против 120) из-за накладных расходов инициализации и освобождения 10 очередей. Накладные расходы на `malloc/free` каждого узла не влияют на подсчёт операций напрямую, но замедляют реальное время исполнения.

2.4. Маршрут программы

1. **Определение имени файла:** из аргумента командной строки или по умолчанию `input.txt`.
2. **Чтение чисел из файла** (`read_file`):
 - первый проход: подсчёт количества чисел n ;
 - выделение массива `malloc(n)`;
 - второй проход: считывание чисел в массив.
3. **Вывод исходного массива** на экран (`print_array`).
4. **Выполнение Radix Sort** (`radix_sort`).
5. **Вывод отсортированного массива** на экран.
6. **Вывод оценки сложности:** n , максимальный элемент, d , k , число операций, число дополнительных узлов.
7. **Освобождение памяти:** `free(arr)`.

3. Описание используемых переменных

В таблице 1 приведено описание переменных программы.

Таблица 1 – Описание переменных программы

Переменная	Тип	Границы	Назначение
Главная программа (main.c)			
filename	const char*	строка	Имя входного файла
arr	int*	адрес	Указатель на массив чисел
n	int	$[1; 2^{31} - 1]$	Количество прочитанных чисел
max_val	int	$[0; 2^{31} - 1]$	Максимальный элемент массива
d	int	$[1; 10]$	Число разрядов max_val
tmp	int	$[1; 2^{31} - 1]$	Временная для подсчёта разрядов
ops	int	$[0; 2^{31} - 1]$	Расчётное число операций
Кольцевая очередь (radix_sort.c)			
Node.value	int	$[0; 2^{31} - 1]$	Хранимое значение узла
Node.next	Node*	адрес	Следующий узел (кольцо)
Queue.tail	Node*	адрес / NULL	Указатель на хвост; tail->next == голова
Queue.size	int	$[0; n]$	Текущее число элементов
node	Node*	адрес	Новый узел при enqueue

Переменная	Тип	Границы	Назначение
head	Node*	адрес	Голова очереди при dequeue
cur, tmp	Node*	адрес	Рабочие указатели в queue_free
Ввод/вывод (main.c)			
fp	FILE*	адрес	Указатель на открытый файл
n	int	$[1; 2^{31} - 1]$	Счётчик при первом проходе
tmp	int	$[0; 2^{31} - 1]$	Временная для подсчёта чисел
arr	int*	адрес	Выделенный массив
i	int	$[0; n - 1]$	Индекс цикла второго прохода
Цифровая сортировка (radix_sort.c)			
exp	int	$\{1, 10, \dots, 10^{d-1}\}$	Текущий разряд
buckets[10]	Queue[10]	—	Массив из 10 корзин-очереди
digit	int	$[0; 9]$	Цифра текущего разряда
idx	int	$[0; n - 1]$	Индекс вставки при сборе
i, j	int	$[0; n - 1], [0; 9]$	Индексы циклов
max	int	$[0; 2^{31} - 1]$	Текущий максимум в find_max
max_val	int	$[0; 2^{31} - 1]$	Максимальный элемент

4. Тестирование программы

4.1. Тест 1: стандартный набор из 20 элементов

Файл `input.txt`:

```
170 45 75 90 802 24 2 66 125 377 40 1 999 312 7 88 456 23 1000 5
```

```
Чтение файла 'input.txt'...
```

```
Прочитано 20 чисел.
```

```
Исходный массив:
```

```
170 45 75 90 802 24 2 66 125 377 40 1 999 312 7 88 456 23 1000 5
```

```
Выполняем Radix Sort...
```

```
Отсортированный массив:
```

```
1 2 5 7 23 24 40 45 66 75 88 90 125 170 312 377 456 802 999 1000
```

```
Оценка сложности
```

n (элементов)	20
max элемент	1000
d (разрядов)	4
k (основание, BASE)	10
Операций:	2361
Доп. ячеек (узлов):	20

Массив корректно отсортирован. Число операций: $7 \cdot 20 + 5 + 4 \cdot (19 \cdot 20 + 174) = 145 + 4 \cdot 554 = 2361$. Дополнительная память — 20 узлов Node (по одному на каждый элемент за один проход).

4.2. Тест 2: малый набор из 8 элементов

Файл `input.txt` по умолчанию:

```
170 45 75 90 802 24 2 66
```

```
Чтение файла 'input.txt'...
```

```
Прочитано 8 чисел.
```

```
Исходный массив:
```

```
170 45 75 90 802 24 2 66
```

```
Выполняем Radix Sort...
```

```
Отсортированный массив:  
2 24 45 66 75 90 170 802
```

```
Оценка сложности  
n (элементов)           8  
max элемент             802  
d (разрядов)            3  
k (основание, BASE)     10  
Операций:               1039  
Доп. ячеек (узлов):     8
```

Число операций: $7 \cdot 8 + 5 + 3 \cdot (19 \cdot 8 + 174) = 61 + 3 \cdot 326 = 1039$.

4.3. Тест 3: несуществующий файл

```
Чтение файла 'missing.txt'...  
Ошибка: не удалось открыть файл 'missing.txt'
```

Программа корректно обрабатывает ошибку открытия файла и завершает работу с кодом EXIT_FAILURE.

4.4. Тест 4: пустой файл

```
Чтение файла 'empty.txt'...  
Ошибка: файл 'empty.txt' пуст или не содержит чисел
```

Программа корректно обнаруживает пустой файл и завершает работу с кодом EXIT_FAILURE.

5. Блок-схемы алгоритма

Блок-схемы всех подпрограмм приведены в Приложении А (стр. 20).
Перечень блок-схем:

1. Главная программа (`main`)
2. Чтение из файла (`read_file`)
3. Цифровая сортировка (`radix_sort`)
4. Сортировка через очереди по разряду (`counting_sort_by_queue`)
5. Добавление элемента в очередь (`enqueue`)
6. Извлечение элемента из очереди (`dequeue`)

6. Исходный код программы

Программа состоит из трёх файлов: `main.c` — точка входа, ввод/вывод и вычисление сложности; `radix_sort.h` — объявления функций; `radix_sort.c` — реализация кольцевой очереди и алгоритма сортировки.

6.1. Файл `main.c`

```
1 #include "radix_sort.h"
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 #define INPUT_FILE "input.txt"
6
7 /*
8  * read_file считывание чисел из текстового файла в динамический массив.
9  * Два прохода: первый считает n, второй заполняет массив.
10 * Итого: 10n + 12 операций (без внутренних операций fscanff).
11 */
12 static int read_file(const char *filename, int **out_arr)
13 {
14     FILE *fp = fopen(filename, "r"); /* +1 вызов, +1 присваивание */
15     if (!fp) { /* +1 сравнение */
16         fprintf(stderr, "Ошибка: не удалось открыть файл '%s'\n", filename);
17         return -1;
18     }
19
20     /* Первый проход: подсчёт чисел */
21     int n = 0, tmp; /* +1 присваивание */
22     while (fscanf(fp, "%d", &tmp) == 1) n++; /* n+1 вызовов, n+1 сравн., n арифм. */
23
24     if (n == 0) { /* +1 сравнение */
25         fprintf(stderr, "Ошибка: файл '%s' пуст или не содержит чисел\n", filename);
26         fclose(fp);
27         return -1;
28     }
29
30     rewind(fp); /* +1 вызов */
31
32     int *arr = (int *)malloc(n * sizeof(int)); /* +1 арифм., +1 вызов, +1 присваивание */
33     if (!arr) { /* +1 сравнение */
34         fprintf(stderr, "Ошибка: не удалось выделить память\n");
35         fclose(fp);
36         return -1;
37     }
38 }
```

```

38
39  /* Второй проход: i=0(+1 присв.); i<n(n+1 сравн.); i++(n арифм.) */
40  for (int i = 0; i < n; i++) {
41      if (fscanf(fp, "%d", &arr[i]) != 1) { /* +1 вызов, +1 инд., +1 сравнение */
42          fprintf(stderr, "Ошибка: неожиданный конец файла\n");
43          free(arr);
44          fclose(fp);
45          return -1;
46      }
47  }
48
49  fclose(fp); /* +1 вызов */
50  *out_arr = arr; /* +1 присваивание */
51  return n; /* +1 выход */
52 }
53 /* Итого: 10n + 12 операций */
54
55 /*
56 * print_array вывод элементов через пробел.
57 * Итого: 7n + 3 операций.
58 */
59 static void print_array(const int *arr, int n)
60 {
61     /* i=0: +1 присв.; i<n: n+1 сравн.; i++: n арифм. */
62     for (int i = 0; i < n; i++) {
63         printf("%d", arr[i]); /* +1 индексация, +1 вызов */
64         if (i < n - 1) /* +1 арифметика, +1 сравнение */
65             printf(" "); /* +1 вызов */
66     }
67     printf("\n"); /* +1 вызов */
68 }
69 /* Итого: 7n + 3 операций */
70
71 int main(int argc, char *argv[])
72 {
73     const char *filename = (argc > 1) ? argv[1] : INPUT_FILE;
74
75     printf("\nЧтение файла '%s'...\n", filename);
76     int *arr = NULL;
77     int n = read_file(filename, &arr);
78     if (n < 0) return EXIT_FAILURE;
79     printf("Прочитано %d чисел.\n", n);
80
81     printf("\nИсходный массив:\n");
82     print_array(arr, n);
83
84     printf("\nВыполняем Radix Sort...\n");
85     radix_sort(arr, n);

```

```

86
87     printf("Отсортированный массив:\n");
88     print_array(arr, n);
89
90     /* Вычисляем d количество разрядов максимального числа */
91     int max_val = find_max(arr, n);
92     int d = 0;
93     int tmp = (max_val == 0) ? 1 : max_val;
94     while (tmp > 0) { d++; tmp /= 10; }
95
96     int ops = 7*n + 5 + d*(19*n + 174);
97     printf("\n Оценка сложности \n");
98     printf("  n (элементов)           %d\n", n);
99     printf("  max элемент           %d\n", max_val);
100    printf("  d (разрядов)           %d\n", d);
101    printf("  k (основание, BASE)      10\n");
102    printf("  Операций:               %d\n", ops);
103    printf("  Доп. ячеек (узлов):      %d\n", n);
104    printf("\n");
105
106    free(arr);
107    return EXIT_SUCCESS;
108 }

```

main: ≈ 18 операций + вызовы функций

6.2. Файл radix_sort.h

```

1 #ifndef RADIX_SORT_H
2 #define RADIX_SORT_H
3
4 int find_max(const int *arr, int n);
5 void radix_sort(int *arr, int n);
6
7 #endif

```

6.3. Файл radix_sort.c

Структуры данных кольцевой очереди

```

1 *           digit=(arr[i]/exp)%BASE: n инд.+n арифм.+n арифм.+n присв.
2 *           &buckets[digit]: n инд.; arr[i]: n инд.
3 *           enqueue: n вызовов; !=0: n сравн. = 11n + 2
4 * Шаг 3 сбор (j = 0..9, while dequeue n)
5 *           idx=0(+1 присв.); j=0(+1 присв.)
6 *           j<10: 11 сравн.; j++ 10: 10 арифм.+10 присв.

```

```

7 *           while-проверка n+10 раз:
8 *           &buckets[j]: n+10 инд.; is_empty: n+10 вызовов;
9 *           !: n+10 сравн.
10 *          dequeue n раз:
11 *           &buckets[j]: n инд.; &arr[idx]: n инд.; dequeue: n вызовов
12 *           idx++ n: n арифм.+n присв. = 8n + 63
13 *      Шаг 4 освобождение 10 очередей (queue_free 10)
14 *           (идентично step 1) = 52
15 *
16 *      Итого counting_sort_by_queue: 19n + 169
17 *
18 * radix_sort (собственный код, нормальный путь):
19 *      n1(+1 сравн.); find_max(+1 вызов+1 присв.); exp=1(+1 присв.)
20 *      max_val/exp>0 (d+1): (d+1)(+1 арифм.+1 сравн.)
21 *      exp*=BASE d: d(+1 арифм.+1 присв.)

```

queue_init: 7 операций: q->tail=NULL (2 ссылки+1 присв.) + q->size=0 (2 ссылки+1 присв.) + 1 выход

queue_is_empty: 4 операции: q->size==0 (2 ссылки+1 сравн.) + 1 выход

Операция enqueue

```

1 * Полностью (с подфункциями):
2 *   find_max:          7n 2
3 *   counting d:        d (19n + 169)
4 *   собственный код:   7 + 5d
5 *
6 *   Итого: 7n + 5 + d (19n + 174)
7 *
8 *   n число элементов, d количество разрядов в max, k = BASE = 10
9 */
10
11 #define BASE 10
12
13 /*
14  * КОЛЬЦЕВАЯ ОЧЕРЕДЬ НА СВЯЗНОМ СПИСКЕ
15  * tail->next == голова; enqueue/dequeue O(1)
16  */
17
18 typedef struct Node {
19     int value;
20     struct Node *next;

```

enqueue: 29 операций (не пустая очередь); 20 операций (пустая)

Операция dequeue

```
1
2 typedef struct {
3     Node *tail;
4     int size;
5 } Queue;
6
7 /*
8  * queue_init обнулить поля очереди
9  * */
10 static void queue_init(Queue *q)
11 {
12     q->tail = NULL; /* +2 ссылки (->tail), +1 присваивание */
13     q->size = 0; /* +2 ссылки (->size), +1 присваивание */
14 } /* +1 выход из метода */
15 /* Итого: 7 операций */
16
17 /*
18  * queue_is_empty вернуть 1 если очередь пуста
19  * */
```

dequeue: 28 операций (много элементов); 24 операции (один элемент)

Освобождение очереди

```
1 {
2     return q->size == 0; /* +2 ссылки (->size), +1 сравнение, +1 выход */
3 }
4 /* Итого: 4 операции */
5
6 /*
7  * enqueue добавить value в хвост очереди, O(1)
8  *
9  * Нормальный путь не пустая очередь:
10  * malloc+присв.(2) + сравн.(1) [node, !node]
11  * ссылки(2)+присв.(1) [node->value=value]
12  * ссылки(2)+сравн.(1) [q->size==0, false]
13  * ссылки(2)+ссылки(4)+присв.(1) [node->next=q->tail->next]
14  * ссылки(4)+присв.(1) [q->tail->next=node]
15  * ссылки(2)+присв.(1) [q->tail=node]
16  * ссылки(2)+арифм.(1)+присв.(1) [q->size++]
17  * выход(1)
18  * = 29 операций
19  *
```

queue_free: $6n + 16$ операций ($n \geq 1$); 4 операции (пустая очередь)

Цифровая сортировка

```
1 {
2     Node *node = (Node *)malloc(sizeof(Node)); /* +1 вызов, +1 присваивание */
3     if (!node) {                               /* +1 сравнение */
4         fprintf(stderr, "Ошибка: malloc вернул NULL\n"); /* +1 вызов */
5         return -1;                             /* +1 выход */
6     }
7
8     node->value = value; /* +2 ссылки (->value), +1 присваивание */
9
10    if (q->size == 0) {                          /* +2 ссылки (->size), +1 сравнение */
11        node->next = node;                      /* +2 ссылки (->next), +1 присваивание */
12    } else {
13        node->next = q->tail->next; /* +2 ссылки (node->next), +4 ссылки (q->tail->
14        ↪ next), +1 присваивание */
15        q->tail->next = node;          /* +4 ссылки (q->tail->next), +1 присваивание */
16    }
17    q->tail = node; /* +2 ссылки (->tail), +1 присваивание */
18    q->size++;      /* +2 ссылки (->size), +1 арифметика, +1 присваивание */
19    return 0;      /* +1 выход */
20 }
21
22 /*
23  * деQUEUE снять с головы очереди, O(1)
24  *
25  * Нормальный путь много элементов:
26  *   ссылки(2)+сравн.(1)                [q->size==0, false]
27  *   ссылки(4)+присв.(1)                [head=q->tail->next]
28  *   ссылка(1)+ссылки(2)+присв.(1)      [*out=head->value]
29  *   ссылки(2)+сравн.(1)                [head==q->tail, false]
30  *   ссылки(4)+ссылки(2)+присв.(1)      [q->tail->next=head->next]
31  *   вызов(1)                           [free(head)]
32  *   ссылки(2)+арифм.(1)+присв.(1)      [q->size--]
33  *   выход(1)
34  *   = 28 операций
35  *
36  * Нормальный путь один элемент:
37  *   вместо (4+2+1): ссылки(2)+присв.(1) [q->tail=NULL]
38  *   = 24 операции
39  * */
40 static int dequeue(Queue *q, int *out)
41 {
42     if (q->size == 0) {                      /* +2 ссылки (->size), +1 сравнение */
43         fprintf(stderr, "Ошибка: очередь пуста\n"); /* +1 вызов */
44         return -1;                          /* +1 выход */
45     }
```

```

46
47 Node *head = q->tail->next;      /* +4 ссылки (q->tail->next), +1 присваивание */
48 *out = head->value;              /* +1 ссылка (*out разым.), +2 ссылки (->value), +1 n
   ↪ присваивание */
49
50 if (head == q->tail) {           /* +2 ссылки (q->tail), +1 сравнение */
51     q->tail = NULL;              /* +2 ссылки (->tail), +1 присваивание */
52 } else {
53     q->tail->next = head->next; /* +4 ссылки (q->tail->next), +2 ссылки (head->next),
   ↪ +1 присваивание */
54 }
55 free(head); /* +1 вызов */
56 q->size--; /* +2 ссылки (->size), +1 арифметика, +1 присваивание */
57 return 0; /* +1 выход */
58 }
59 /* Итого (много элементов): 28 операций; (один элемент): 24 операции */
60
61 /*
62 * queue_free освободить все узлы очереди
63 *
64 * Пустая очередь: ссылки(2)+сравн.(1)+выход(1) = 4
65 * n 1:
66 *   ссылки(2)+сравн.(1) [q->size==0, false] = 3
67 *   ссылки(4)+присв.(1) [head=q->tail->next] = 5
68 *   присв.(1) [cur=head] = 1
69 *   тело n:
70 *     присв.(1) [tmp=cur]
71 *     ссылки(2)+присв.(1) [cur=cur->next]
72 *     вызов(1) [free(tmp)]
73 *     сравн.(1) [cur!=head]
74 *     итого за итерацию: 6 6n
75 *     ссылки(2)+присв.(1) [q->tail=NULL] = 3
76 *     ссылки(2)+присв.(1) [q->size=0] = 3
77 *     выход(1) = 1
78 * */
79 static void queue_free(Queue *q)

```

find_max: $7n - 2$ операций (худший случай)

counting_sort_by_queue: $19n + 169$ операций ($k=10$)

radix_sort: $7n + 5 + d \cdot (19n + 174)$ операций итого

Заключение

В ходе выполнения лабораторной работы была разработана программа на языке C, реализующая цифровую сортировку (Radix Sort LSD) целых неотрицательных чисел с использованием кольцевой очереди на основе односвязного списка.

Программа организована в три файла: `main.c` содержит точку входа, ввод/вывод и вывод оценки сложности; `radix_sort.h` объявляет интерфейс; `radix_sort.c` реализует кольцевую очередь и алгоритм Radix Sort.

Вместо вспомогательного массива `output[]` и счётчиков `count[]` используются 10 кольцевых очередей-корзин: на каждом проходе элементы распределяются по корзинам через `enqueue`, затем собираются обратно через `dequeue`. Порядок FIFO обеспечивает устойчивость сортировки без дополнительных условий.

Теоретическая сложность алгоритма составляет $O(d \cdot n)$ по времени и $O(n)$ по памяти. При фиксированном d алгоритм работает за **линейное** время $O(n)$. По сравнению с реализацией через массив (ЛР 1, $26n + 120$ на проход) линейный коэффициент ниже ($19n$ против $26n$), однако константа выше (169 против 120) из-за накладных расходов инициализации и освобождения 10 очередей на каждый проход. Асимптотика остаётся той же. Результаты тестирования подтверждают корректность сортировки и правильную обработку ошибочных ситуаций.

Приложение А. Блок-схемы алгоритма

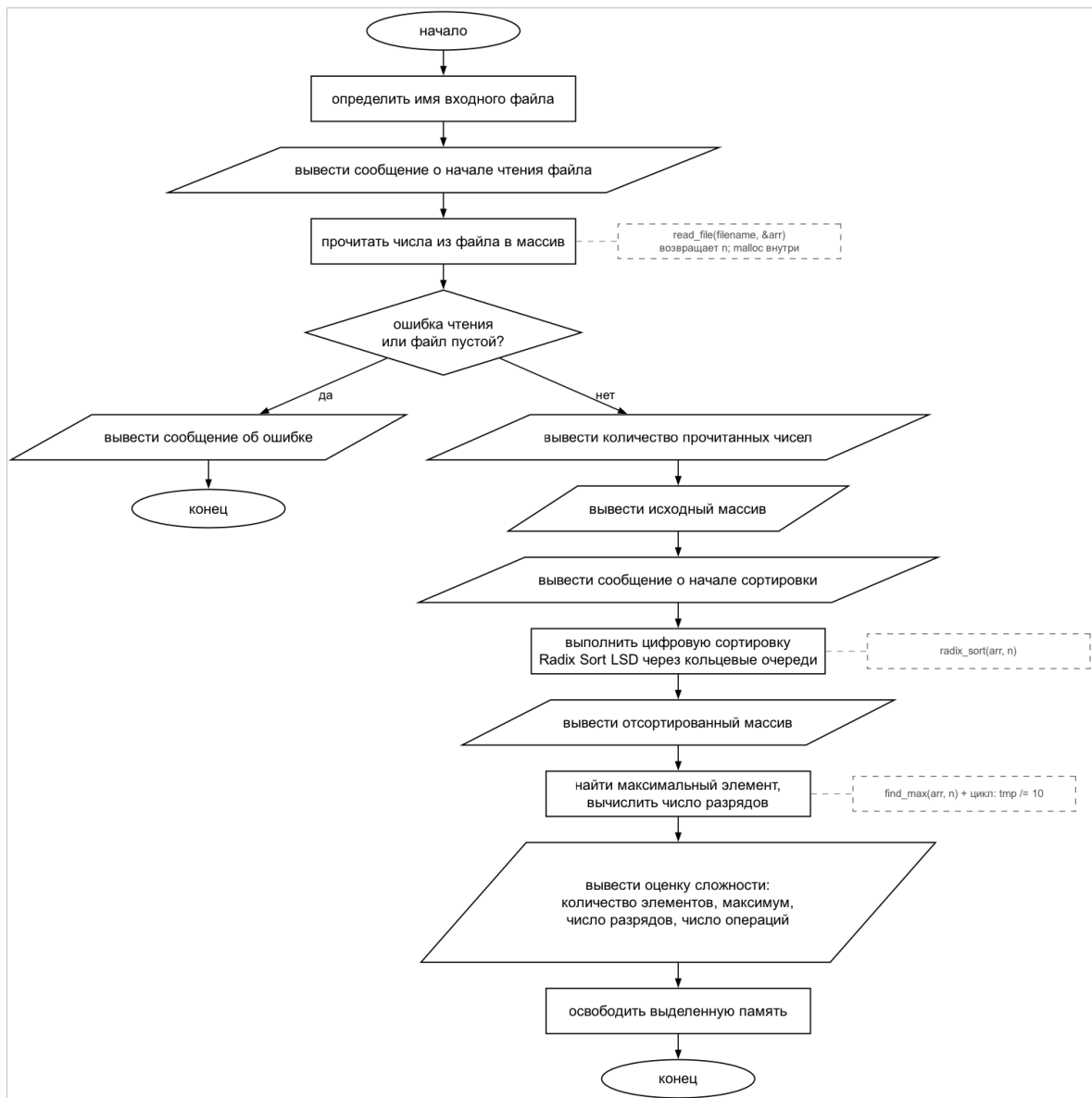


Рис. 1 – Главная программа (main)

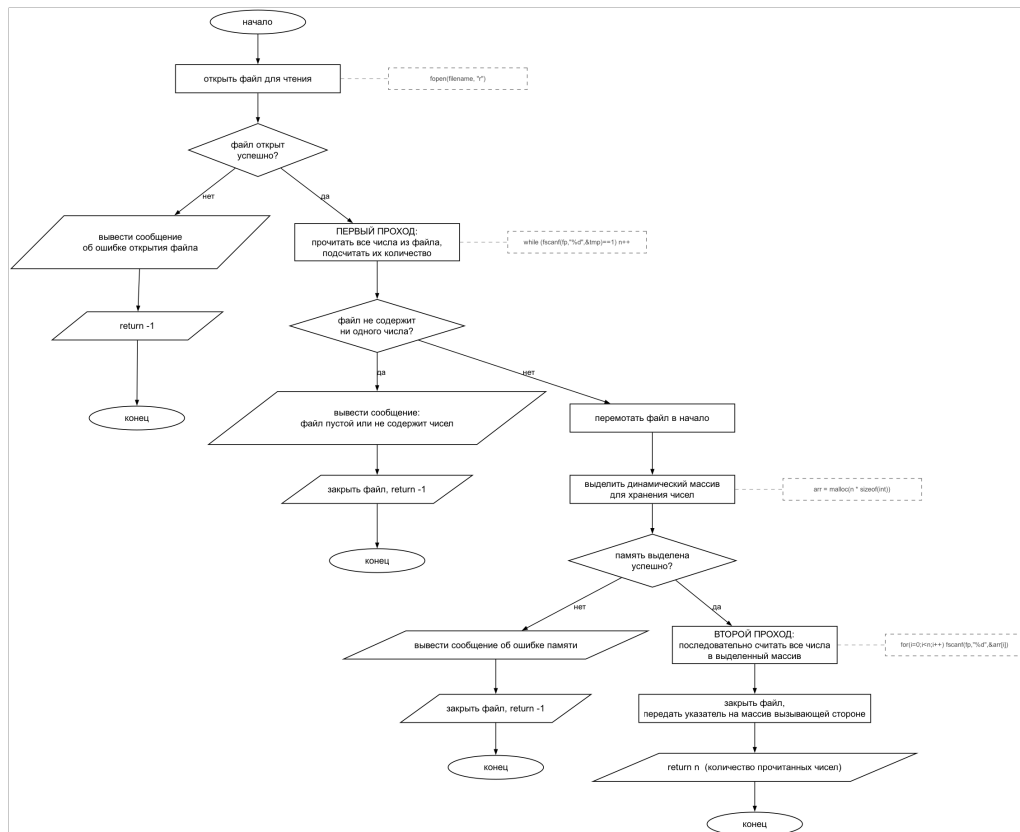


Рис. 2 – Чтение из файла (read_file)

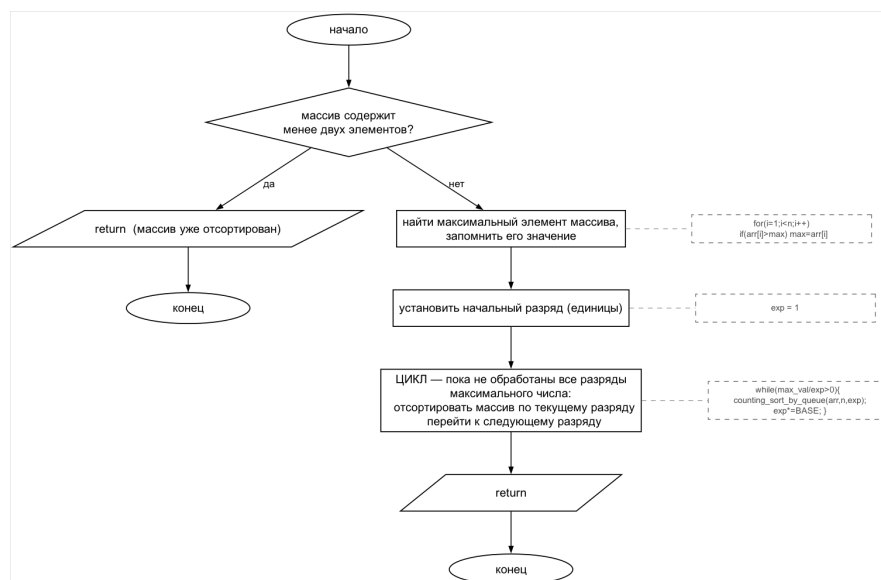


Рис. 3 – Цифровая сортировка (radix_sort)

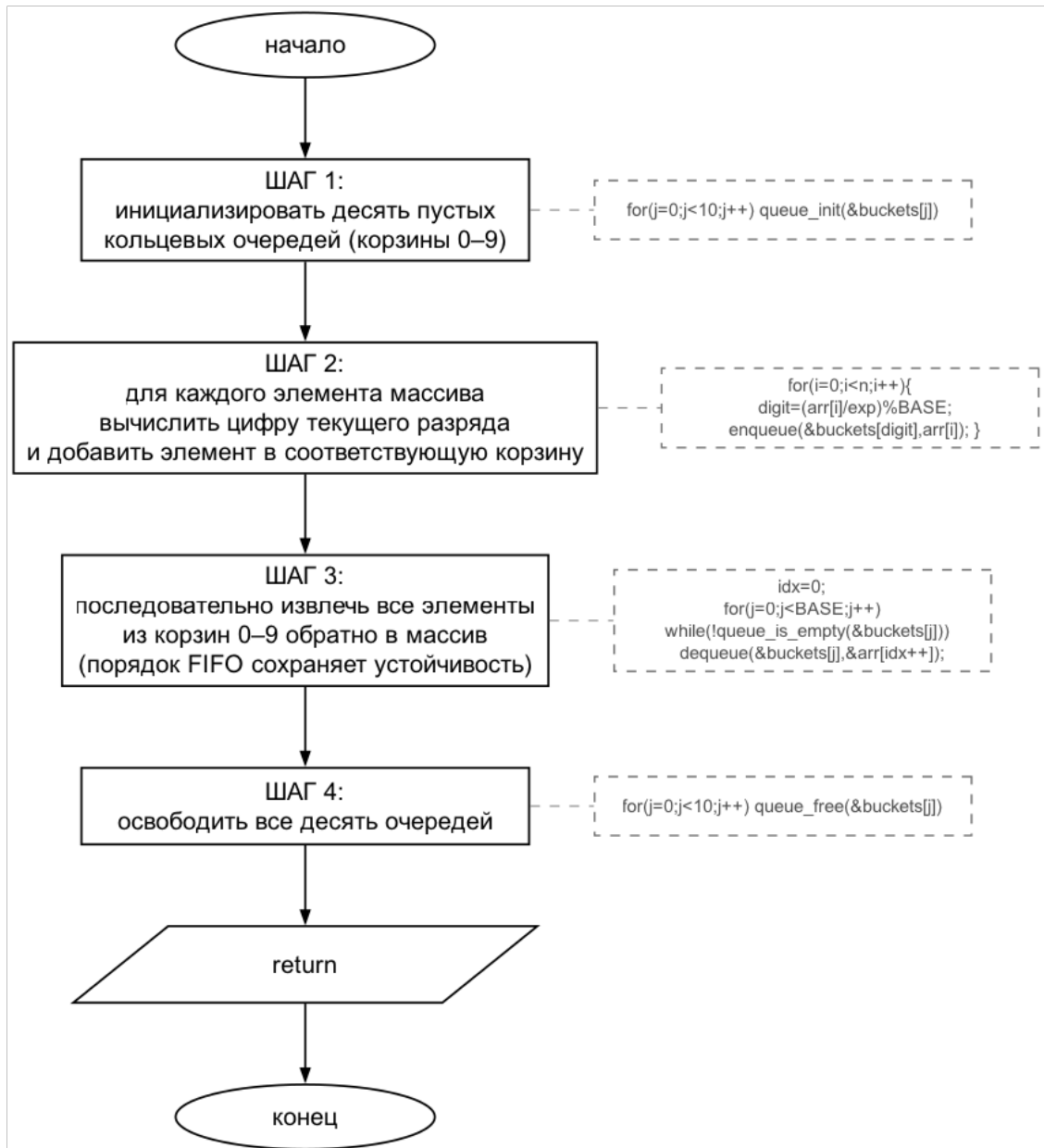


Рис. 4 – Сортировка через очереди по разряду (counting_sort_by_queue)

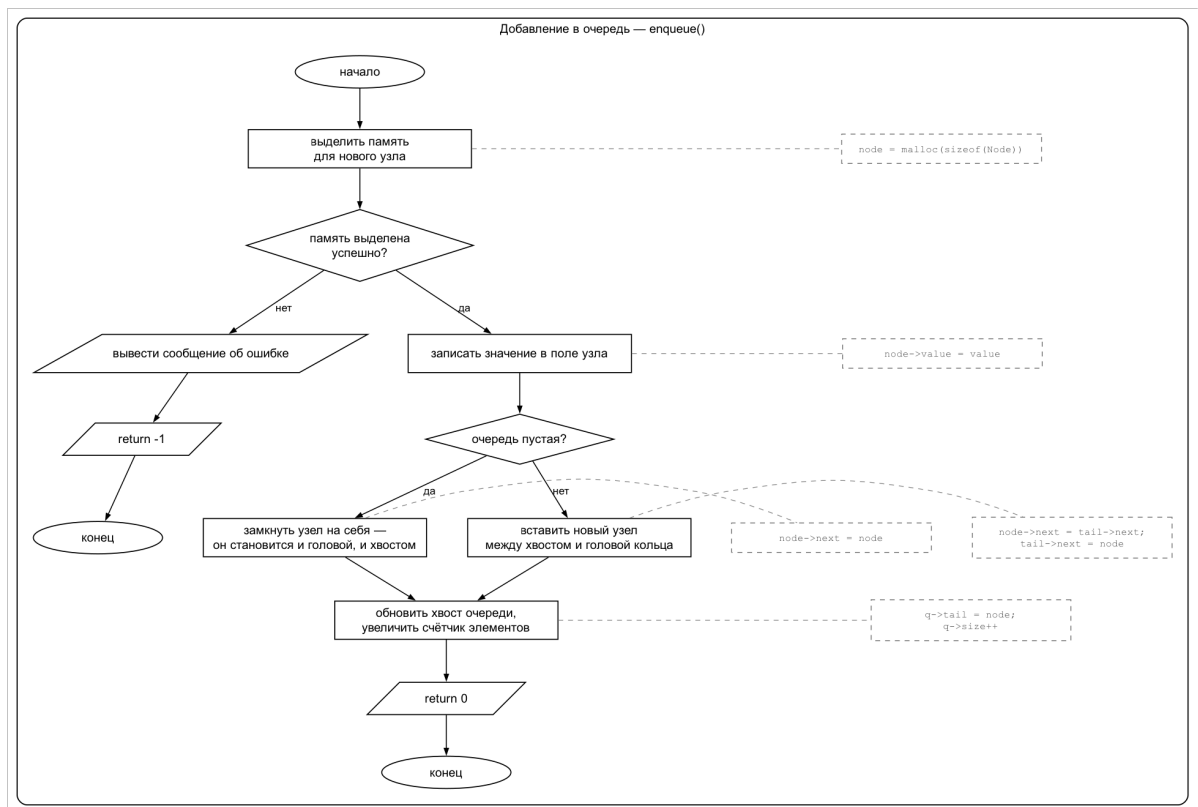


Рис. 5 – Добавление элемента в очередь (enqueue)

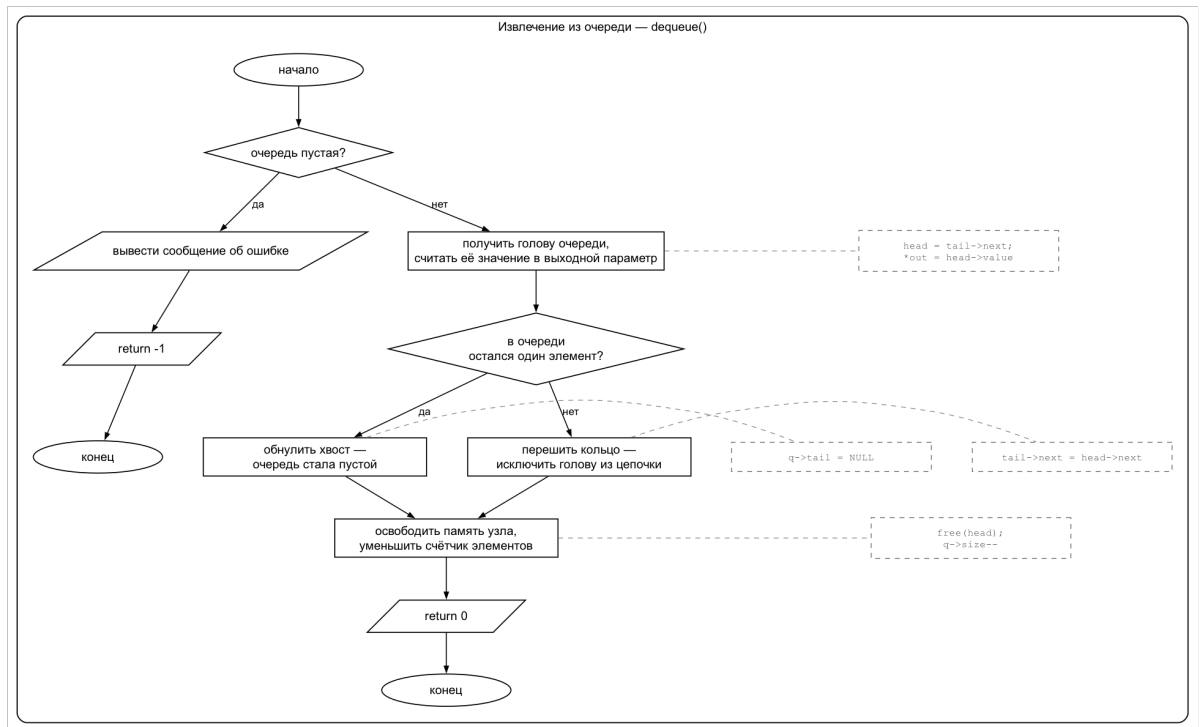


Рис. 6 – Извлечение элемента из очереди (dequeue)